

Seconds, Not Blocks:

A Validated Speed-of-Light for Offloaded Mixture-of-Experts Decode, and What It Says About Published Speed-Ups

Harshith Kantamneni
Independent Researcher

Abstract

Running a large Mixture-of-Experts (MoE) model on one consumer GPU means keeping most experts in host memory, and dozens of systems now claim faster decoding in this setting, often against llama.cpp. These claims are stated as speed-ups over baselines whose configuration is rarely reported, and the evaluation contracts proposed so far count cache blocks, not seconds. We build the missing yardstick and use it. **(1)** A lower bound on batch-1 decode time, in seconds, for any policy that executes the model’s exact routing within a per-layer or a shared all-layer GPU expert budget, computed from exact routing traces. **(2)** A bytes-over-bandwidth decode model, validated out of sample on 52 third-party measurements from 9 sources (16% median error, leave-one-group-out) and tested on pre-registered first-party and anchor runs, where its transferred constants proved conservative in the offloaded regime. **(3)** An audit of 147 published measurements from 41 systems: after an independent re-check against every source, 22 are adjudicable. None of the 15 papers with a llama.cpp baseline configured its partial expert offload (`--n-cpu-moe`); 16 of 20 adjudicated llama.cpp baselines fall below the band of the model’s equal-memory prediction, and 8 of 22 claimed gains survive against it. The median adjudicated system reaches 24% of the physical speed-of-light. **(4)** A trace-provenance study on 9 models: routing traces taken by teacher-forcing dataset text, the norm in this literature, overstate cache hit rates at small budgets against the models’ own generations on every model (by 3.3–5.4 points on the 4 pre-registered models, by up to 8.8 on the audit’s models, two of which cross our registered threshold for a changed conclusion), while the bound on our platform does not move; and for gpt-oss-120b, dataset text without the model’s own reasoning is off-distribution (NLL 6.40 vs 3.00 nats with its reasoning inserted), reproduced in three implementations. We also report our own measurement mistakes, and turn all of it into a time-level reporting contract.

1 Introduction

Batch-1 decode of an MoE model with experts in host memory is a data-movement problem: every token reads the routed experts’ weights from wherever they are, GPU

memory, host DRAM or across PCIe [10, 6]. The systems literature has answered it with mechanisms: GPU expert caches with demand fetch [4, 25, 18], CPU execution of misses [12, 2, 29, 8, 26], expert prediction [9, 30, 23], speculative decoding with prefetch [3, 21]. They report speed-ups over baselines of their choosing (median $1.4\times$ and up to $4.2\times$ in the rows we audit); 15 of the 35 papers we audit include llama.cpp [5], the engine desktop users actually run.

Two things are missing. First, a ceiling: no paper we found states how fast its hardware *could* decode the model within the same GPU memory, so a $2\times$ speed-up cannot be told apart from closing half the gap or a tenth of it. Second, baseline strength: llama.cpp can place experts per layer (`--n-cpu-moe`, since mid-2025) as well as whole layers (`-ngl`), and the difference at equal memory is large. The most careful methodology paper in this area proposes a trace-level evaluation contract but works in cache blocks and explicitly leaves “absolute transferred bytes per output token together with the bandwidth budget” unfilled [28].

This paper fills that field. We state the bound in seconds, validate the model that computes it three ways, and use it to re-express published results as fractions of what their hardware allows, next to the baseline they should have run. Following Hoeffler and Belli [7], we pre-registered hypotheses and predictions before measuring, and we report where they failed (Appendix B) and the mistakes in our own earlier measurements that the process exposed (Section 9).

Contributions.

- **A speed-of-light in seconds** (Section 4) for any exact-routing policy with a per-layer or a shared all-layer expert budget, static or dynamic, demand or prefetching, with misses fetched or executed on the CPU, computed from exact routing traces taken without a GPU-resident model (Section 3), on 7 architectures.
- **A three-way validation of the decode model** (Section 5): third-party measurements, pre-registered first-party runs, and pre-registered anchors.
- **An audit of published MoE-offloading results** (Section 7) against the bound and against a predicted

equal-memory llama.cpp baseline, with every row re-checked against its source by an independent pass.

- **Trace provenance** (Section 6): what teacher-forcing dataset text does to locality, to the bound, and, for one model, to the model’s state.
- **A reporting contract in seconds** (Section 10), informed by the pitfalls we hit (Section 9).

Code, traces (2.65M answer tokens over 9 models and three text arms), the 147-row audit set with quoted evidence, job scripts, raw measurements and all pre-registrations are released; git history orders every prediction before the measurement it predicts.

2 Background and Related Work

Offloading systems. Demand-paging systems keep a per-layer GPU cache and copy misses over PCIe [4, 25, 18, 20]. CPU-execution systems run non-resident experts on the host [12, 2, 22, 29, 8], and FreeToken chooses per miss between the two with a closed-form split [26]. Predictors guess upcoming experts from the previous layer’s state [9], from pre-attention activations [30], or several steps and layers ahead [23]; speculative systems prefetch the experts a draft will need [3, 21]. Batched-throughput systems [1, 27] and SSD tiers [13, 14] are outside our scope (single request, host DRAM).

Evaluation methodology. Zhang [28] shows that replay semantics, workload contamination and operating regime change which caching policy wins, and proposes a trace-level contract with a Belady gap; every result there is a block count, and it declines to relabel published speed-ups. MoE-CAP [11] defines sparsity-aware bandwidth utilisation against a single peak, which is ill-defined when expert bytes come from three memories. “Cacheable by Design?” [19] matches bytes over bandwidth to one configuration. Si et al. [17] derive the reuse count at which a copy pays off. Qiu et al. [16] predict single-sequence llama.cpp throughput from GGUF metadata with roofline-shaped models (13–36% error on three hosts), for models run fully on one device; our model covers the offloaded configurations and serves as the base of a bound. Hoeffler and Belli’s rules [7], in particular “show upper performance bounds”, frame our approach; the roofline [24] is its single-memory ancestor.

Our earlier system. We previously built a GPU expert cache with CPU-executed misses into llama.cpp. A novelty check afterwards found the mechanism taken [29, 8, 26, 2, 15]; we keep it here only as a system under test (Section 8).

3 Exact Routing Traces

In a causal decoder the hidden state of token t depends only on tokens 1.. t , so the experts that decode selects

for token t are those that teacher-forced prefill selects at position t , up to floating-point ties. Prefill lets us invert the loop order: load one decoder layer, push the whole corpus through it, record the router’s top- k , free the layer. Weights are read once per corpus and peak memory is one layer, so a 2-vCPU VM traces a 120B model by reading tensors over HTTP range requests, and a GPU traces a corpus in minutes.

Exactness. On tiny random checkpoints of 7 architectures (Qwen3-MoE, Qwen2-MoE, OLMoE, DeepSeek-V2, Mixtral, PhiMoE, gpt-oss with MXFP4 and bf16 experts), the collector’s selections equal those of the reference **transformers** model run as prefill *and* as token-by-token decode with a KV cache, and the per-token loss agrees within 10^{-3} nats, on CPU and on GPU. On real models, fp32 traces taken on a GPU differ from CPU traces in at most 0.07% of expert selections (near-ties), and on the real Mixtral-8x7B, Phi-3.5-MoE and Qwen2-57B checkpoints the stock **transformers** model, run whole on a CPU, picks the same experts as the collector on a GPU for all but 0.35% of (token, layer) pairs (6,717 tokens).

Corpora and provenance. Prompts are 160 two-turn conversations from four public datasets (chat, math, code, multilingual), formatted with each model’s own chat template. For each of 9 models (OLMoE-1B-7B, gpt-oss-20b and -120b, Qwen3-30B-A3B, and the audit’s Mixtral-8x7B, DeepSeek-V2-Lite, Qwen1.5-MoE-A2.7B, Qwen2-57B-A14B and Phi-3.5-MoE, chat or instruct variants) we trace three arms over the same prompts (for gpt-oss-120b, G and S cover the first 35): **D**, the dataset’s reference answer, teacher-forced, as in most trace studies; **G**, the model’s own greedy answer; **S**, the model’s own answer sampled with its generation-config defaults (seed = prompt index), each up to 1024 new tokens, generated with vLLM. Decode steps are the answer tokens only, replayed back to back as one session.

4 A Speed-of-Light in Seconds

Decode model. For one decode step,

$$T = \frac{D + X_g}{\eta_g B_g} + \frac{X_c}{\eta_c B_c} + n_c \tau_e + \frac{X_f}{\eta_p B_p} + n_f \tau_x, \quad (1)$$

with D the dense per-token bytes (attention, shared experts, routers, norms, head, KV reads), X_g, X_c, X_f routed-expert bytes read from GPU memory, executed on the CPU (n_c executions) and fetched over PCIe (n_f transfers), B datasheet bandwidths, η efficiencies and τ latencies. Bytes are exact: every parameter is classified on PyTorch’s meta device, or read from GGUF headers. For llama.cpp’s `--n-cpu-moe` n , X_c is the selected experts of the first n MoE layers and $n_c = nk$.

Bound. Let a policy keep at most C experts of each layer on the GPU (per-layer budget), or at most LC over all layers (global budget). Each routed expert execution runs from a resident copy on the GPU (cost a) or on the CPU from host memory (cost b); each resident copy costs one load over PCIe (p), which may overlap anything.

Proposition 1. *Let M^* be the misses of Belady’s MIN with bypass on the routing trace under the same budget (per layer, or on the interleaved all-layer stream for a global budget), N the layer-steps, and $\ell(m) = \max((k-m)a, mb)$. For any exact-routing policy under that budget, the mean time per token satisfies*

$$\bar{T} \geq \min_{x \geq 0} \max \left(T_D + L \min_{m \geq (M^*/N - x)^+} \ell(m), Lxp \right). \quad (2)$$

Proof sketch. Group each expert’s requests into maximal runs served by one resident copy; a run of r requests needs one load and bridges $r - 1$ reuse gaps. MIN with bypass maximises bridged gaps under the budget, and a prefetching schedule bridges no more gaps than some demand schedule with bypass, so GPU executions $\leq G^* +$ loads, i.e. CPU executions $\geq M^* - xN$ with x loads per layer-step. Per layer-step the time is at least $\ell(m)$ when CPU and GPU run concurrently; ℓ is convex, so by Jensen the mean over layer-steps is bounded at the mean m . The link must carry the loads: $\bar{T} \geq Lxp$. Hand-offs, latencies and the capacity cost of prefetching are dropped, so the bound stays valid. $\square$

Two consequences. (i) The bound is often *independent of the budget*: when M^*/N is below $ka/(a+b)$, the optimum runs a share of even resident experts on the CPU, so the speed-of-light is CPU+GPU bandwidth aggregation, above all-GPU speed. (ii) A shared all-layer budget can only lower M^* ; on our traces the global variant is 3.6–7.2% lower in misses at 12.5% of experts (Table 1). We test Proposition 1 on a Qwen3-30B-A3B trace (its first 12 layers tiled to 48; 4,000 steps for the global budget), on six platforms and four parameter sets, against LRU, LFU, MIN, MIN-bypass, our online admission policy and an oracle prefetcher per layer, and against LRU, LFU, MIN and MIN-bypass on the shared budget: no policy beats it; the tightest reaches 98.4% of it per layer and 94.8% globally. *Physical* speed-of-light sets every $\eta = 1$ and every $\tau = 0$ with datasheet peaks; the *implementation-relative* version uses the GPU efficiency of a measured all-GPU run.

5 Validating the Model

Third-party. On 52 published measurements from 9 independent sources (CPU-only, static offload in four engines, all-in-VRAM, demand fetch; 8 models), cross-validation that holds out one of 12 groups at a time (one source’s demand-fetch rows are split by GPU) gives 16% median error (MAPE 17%; static-offload rows 20%). The one term the data support beyond bytes over bandwidth is

a per-CPU-expert latency ($\tau_e = 21 \mu\text{s}$). The model variant was chosen by the same cross-validation, so these errors are slightly optimistic.

First-party, pre-registered. On an NVIDIA A10 (24 GB, PCIe 4, 30-vCPU Xeon 8358 VM) we registered predictions before measuring (H0–H4, Appendix B). No configuration beat the roofline, and decode time under `--n-cpu-moe` is affine in n for every model ($R^2 \geq 0.996$), as Eq. (1) requires. The registered accuracy test failed: with constants transferred from third-party data every prediction was too slow, 27% median error against 20% registered. On the later llama.cpp runs of our case study the transferred constants under-predict by 2–35%; Section 7 discusses what this conservatism means for the audit. Post hoc, fixing η_g and η_c from two gpt-oss-20b runs brought the median error to 4.9% over 29 configurations; we registered that procedure for the next platform (H5) but have not run it.

Anchor, pre-registered. We registered anchor criteria before any anchor run (protocol 4.7): no configuration may beat the physical floor (A1); on an A100 x86 host the median error must be at most 25% (A2) and the model must under-predict at least two thirds of the configurations, as on the A10 (A3). The predictor is the third-party fit with each machine’s STREAM Triad and datasheet GPU bandwidth; each prediction file was committed by the job before the one that measured. No A100 had capacity during the study, so the registered A100 test is still open. On a GH200 (72-core Grace, LPDDR5X, NVLink-C2C; Triad 301 GB/s), a platform outside the fitted envelope, 29 `--n-cpu-moe` configurations of six GGUF files (Mixtral-8x7B at Q4_K_M and Q8_0; Phi-3.5-MoE, Qwen2-57B-A14B and Qwen3-30B-A3B at Q4_K_M; DeepSeek-V2-Lite at Q8_0) show two regimes. With experts on the CPU (25 configurations) the model under-predicts 20 of them, measured/predicted 0.55–1.50 (median error 23%), the conservative direction seen on the A10. All-GPU runs (4) are over-predicted, measured/predicted 0.28–0.92: at 4 TB/s a small model’s decode step is dominated by per-layer fixed costs that Eq. (1) omits. Over all 29, the median error is 24%; no configuration exceeds the physical floor (the closest reaches 86% of it), and time per token is affine in the CPU layers ($R^2 \geq 0.997$). The audited rows sit in the offloaded regime, where the model errs on the side of calling baselines strong.

6 Trace Provenance

Trace studies in this area, including our own earlier work, teacher-force dataset text (arm D). We registered what would count as provenance changing a conclusion before tracing the models’ own text: a reversal of two policies’ order with CIs excluding zero in both arms (F1), a change of ≥ 5 points in DFA’s hit rate at 25% (F2), of $\geq 5\%$ in

the bound (F3), or of ≥ 5 points in a measured system’s fraction of the bound (F4).

Result. None of F1–F4 fires on any registered model (Table 1; F3 and F4 need a measured all-GPU run and were computed for gpt-oss-20b and Qwen3): by the registered rule, provenance is benign at this resolution. The direction, however, is the same everywhere: the models’ own answers are *less* local than dataset answers. At 12.5% of experts, the deployed policy’s hit rate falls by 3.3–5.4 points and MIN-bypass by 1.8–2.9 (every 95% CI excludes zero; paired bootstrap over prompts); at 50% it is within 2.1 points. We had predicted the opposite for greedy text (P2), and were wrong on 3 of 4 models. The bound does not move, because on our platform it sits at the bandwidth-aggregation optimum, and a measured system’s fraction of it moves by at most 1.4 points. Dataset traces are therefore safe for bounds but optimistic for hit rates at small budgets, which is where offloading papers operate. The audit’s five models, traced afterwards with the same script (an extension outside the registration), agree in sign on all 5: DFA’s hit rate at 12.5% is 1.0–8.8 points lower on own text (CI excluding zero on 5 of 5), and on two of them (DeepSeek-V2-Lite and Phi-3.5-MoE) the change at 25% exceeds the 5-point threshold we had registered as changing a conclusion (F2); on these models the gap persists at 50% (up to 4.5 points). Across all 9 models the direction never reverses.

When dataset text is off-distribution. gpt-oss-120b’s teacher-forced loss on dataset answers is near-uniform on some conversations. We classified this before measuring (registered classes: implementation, user-text, format, genuine). On the 35 traced conversations vLLM and our fp32 collector agree (6.40 and 6.39 nats per answer token; mean per-token difference 0.62 nats on the affected conversations, below the registered 1 nat), and llama.cpp gives 5.85 on the 12 A10 test answers, so it is not an implementation artifact. It is not user text: on answer tokens alone, 120b’s loss is $3.00\times$ gpt-oss-20b’s. Inserting the model’s own greedy reasoning (the **analysis** channel, which 120b produced on 30 and 20b on 21 of the 35 prompts) before the answer brings it to 3.00 nats, $1.48\times$ the 20b’s under the same format (registered threshold $1.5\times$); on the 21 conversations where both models reasoned, the ratio falls from $2.78\times$ to $1.12\times$. A “low reasoning” system prompt does nothing (6.56). The larger model, forced to answer without reasoning, assigns human answers far lower probability than the smaller one does: dataset text in the final channel is off-distribution for it, and its routing there is not what deployment produces. Teacher-forced traces of reasoning models should include the model’s own reasoning.

7 Auditing Published Speed-Ups

Rows. Three extraction passes, run by AI agents (one per group of papers), collected 147 measurements from 41 systems published between 2023 and September 2026 (35 papers and 6 llama.cpp pull requests, forks and discussions), each with a verbatim quote or table row and its location, so that every value can be checked by a person. Values that exist only in figures were read from the vector drawing or pixel-measured and flagged. We admitted rows under rules fixed before any audit number was computed: single-request decode, experts partly in host DRAM (no SSD tier, one GPU), the model, quantisation, GPU and host named, and at most two of {DRAM configuration, PCIe generation, expert budget} imputed as a band. 58 rows qualified. Three further agent passes, which had not seen the extraction, re-read the sources of all 58 rows field by field and asked for corrections to 13; applying them, and two rules we then applied uniformly (FreeToken’s measured host bandwidth and context), changed 19 rows, and we excluded one that one checker flagged as effectively all-in-VRAM (`data/audit/verification.md` lists every change). 52 rows have a model we can account for byte by byte.

Method. For each row we compute (i) the physical speed-of-light for its model, budget and hardware, with M^* from our on-policy traces where we have the model (29 rows) and from independent routing otherwise; (ii) the *predicted equal-memory baseline*: llama.cpp `--n-cpu-moe` with the MoE layers that do not fit the row’s expert budget on the CPU, from Eq. (1) refitted with the row’s own system left out; its band combines the model’s cross-validated 10th–90th percentile error ($0.78\text{--}1.31\times$) with any imputation band, and an imputed budget contributes its whole range down to no experts on the GPU. A reported baseline below the band is *weak*; a normalised speed-up S_n (system over predicted baseline) whose band lies above 1 *survives*; rows whose band spans more than $\pm 40\%$, that claim less than $1.2\times$ or that report no baseline are not adjudicated. No row is called wrong. Two analysis choices were fixed after a first pass over the rows and before this final one (protocol 4.7): a row is labelled by its strongest reported llama.cpp baseline, which can only reduce weak calls, and M^* comes from our new traces of the audit’s models.

Findings. (1) **No paper configured llama.cpp’s partial expert offload as its baseline.** 15 of the 35 papers report a llama.cpp baseline; where its configuration is stated, llama.cpp ran whole layers on the CPU (`-ngl`), all experts on the CPU (`-cmoe`), a custom fork, or through Ollama; most papers state neither version nor flags. Only two community threads compared against llama.cpp’s automatic placement (`--fit on`), which does split experts per layer. (2) **16 of 20 adjudicated llama.cpp baselines are weak** (below the band); 10 of them reach at

Table 1: Trace provenance: own sampled text (S) against dataset text (D), response tokens only, paired bootstrap 95% CIs over prompts. Hit rates in points at 12.5% / 25% of experts; DFA is our deployed admission policy, MIN-bypass the oracle. ΔM_{glob}^* : reduction of MIN-bypass misses (S) when the per-layer budget is pooled over all layers, at 12.5%. Rows below the rule extend the registered study to the audit’s models. NLL (nats per answer token) from vLLM on the generation checkpoint (FP8 for Qwen3, Mixtral, Qwen2-57B and Phi-3.5; bf16 otherwise); gpt-oss-120b’s D value covers all 160 conversations, its G and S arms 35. [†]F3 needs a measured all-GPU run and was computed for gpt-oss-20b and Qwen3 only.

model	E/k	NLL D	NLL S	Δ DFA 12.5%	Δ DFA 25%	Δ MIN-b. 12.5%	ΔM_{glob}^*	F1–F3
OLMoE-1B-7B	64/8	1.10	0.63	-3.7 [-4.7, -2.5]	-3.7 [-4.7, -2.6]	-2.0 [-2.6, -1.4]	3.6%	none [†]
gpt-oss-20b	32/4	2.42	1.04	-5.4 [-7.2, -3.5]	-3.3 [-4.9, -1.7]	-2.9 [-3.8, -1.9]	5.4%	none
Qwen3-30B-A3B	128/8	1.73	0.17	-3.3 [-4.8, -1.6]	-1.4 [-2.4, -0.2]	-2.1 [-2.8, -1.2]	5.4%	none
gpt-oss-120b	128/4	6.60	0.48	-5.3 [-7.6, -3.2]	-3.4 [-4.8, -1.9]	-1.8 [-3.1, -0.6]	6.3%	none [†]
Mixtral-8x7B	8/2	1.53	0.36	-2.1 [-2.7, -1.4]	-3.1 [-4.1, -2.1]	-0.9 [-1.1, -0.6]	6.4%	none [†]
DeepSeek-V2-Lite	64/6	1.74	0.18	-8.8 [-11.1, -6.2]	-7.9 [-10.2, -5.4]	-4.6 [-5.7, -3.4]	3.8%	F2
Qwen1.5-MoE-A2.7B	60/4	1.65	0.28	-2.6 [-4.1, -1.2]	-1.7 [-3.4, -0.0]	-2.1 [-2.9, -1.4]	4.3%	none [†]
Qwen2-57B-A14B	64/8	1.33	0.27	-1.0 [-1.3, -0.7]	-1.7 [-2.1, -1.2]	-1.0 [-1.4, -0.7]	4.5%	none [†]
Phi-3.5-MoE	16/2	1.37	0.78	-7.0 [-9.0, -5.0]	-6.9 [-8.7, -5.1]	-4.4 [-5.6, -3.3]	7.2%	F2

Table 2: The 22 adjudicated rows. Claimed: system over its strongest reported llama.cpp baseline (marked $^\ell$), else over its strongest reported baseline. Predicted: equal-memory llama.cpp `--n-cpu-moe` [band]. S_n : system over predicted. SoL: physical speed-of-light, M^* from our traces (t) or independent routing (i). Budget: GPU bytes for routed experts (* imputed upper bound). All rows in Appendix A.

system	model	GPU	budget GB	sys. tok/s	reported base	claimed	predicted [band]	S_n [band]	sys./SoL	label
CPU-GPU collaborative inf.	Phi-3.5-MoE	RTX 4090	19.7	10.4	6.3	1.65	10.3 [7.8, 14.1]	1.01 [0.74, 1.34]	19% (t)	not est.
HybriMoE	DeepSeek-V2-Lite	RTX A6000	1.9	14.5	11.7 ^ℓ	1.24	80.9 [60.8, 109.7]	0.18 [0.13, 0.24]	3% (t)	weak; not est.
HybriMoE	DeepSeek-V2-Lite	RTX A6000	5.6	15.3	8.9 ^ℓ	1.72	140.0 [106.5, 187.6]	0.11 [0.08, 0.14]	2% (t)	weak; not est.
HybriMoE	Mixtral-8x7B	RTX A6000	5.8	4.4	3.5 ^ℓ	1.23	14.6 [10.8, 20.1]	0.30 [0.22, 0.40]	8% (t)	weak; not est.
HybriMoE	Qwen2-57B-A14B	RTX A6000	6.4	9.6	7.4 ^ℓ	1.31	19.1 [14.3, 26.0]	0.50 [0.37, 0.67]	11% (t)	weak; not est.
HybriMoE	Qwen2-57B-A14B	RTX A6000	19.1	9.0	7.2 ^ℓ	1.25	29.8 [22.7, 39.9]	0.30 [0.22, 0.39]	8% (t)	weak; not est.
KTransformers	DeepSeek-V2.5-1210	A100 40GB	0.0	11.9	7.0 ^ℓ	1.70	10.4 [7.9, 14.0]	1.15 [0.85, 1.51]	49% (i)	weak; not est.
KTransformers	DeepSeek-V2.5-1210	RTX 4080	0.0	17.6	9.9 ^ℓ	1.77	12.4 [9.5, 16.6]	1.42 [1.06, 1.85]	55% (i)	at strength; survives
KTransformers	DeepSeek-V3-0324	A100 40GB	0.0	5.9	4.7 ^ℓ	1.25	5.6 [4.2, 7.6]	1.04 [0.77, 1.38]	49% (i)	at strength; not est.
KTransformers	DeepSeek-V3-0324	RTX 4080	0.0	16.1	9.1 ^ℓ	1.78	13.3 [10.1, 17.8]	1.22 [0.91, 1.60]	48% (i)	weak; not est.
KTransformers	Qwen2-57B-A14B	A100 40GB	0.0	22.9	13.0 ^ℓ	1.76	15.5 [11.8, 20.9]	1.47 [1.09, 1.94]	65% (i)	at strength; survives
KTransformers	Qwen2-57B-A14B	RTX 4080	0.0	37.0	19.2 ^ℓ	1.93	19.2 [14.6, 25.6]	1.93 [1.44, 2.53]	77% (i)	at strength; survives
Pipelined sharding	Qwen3-235B-A22B	RTX 5090	0.0	7.7	5.7 ^ℓ	1.35	13.1 [10.2, 17.3]	0.59 [0.45, 0.76]	19% (i)	weak; not est.
Pipelined sharding	Qwen3-235B-A22B	RTX 5090	27.6	11.5	8.5 ^ℓ	1.36	18.2 [14.1, 23.9]	0.63 [0.48, 0.81]	11% (i)	weak; not est.
Pipelined sharding	Qwen3-30B-A3B	RTX 5090	5.7	32.1	26.1 ^ℓ	1.23	69.7 [54.0, 91.6]	0.46 [0.35, 0.59]	3% (t)	weak; not est.
SP-MoE	DeepSeek-V2-Lite	A100 40GB	3.3	5.6	3.7	1.50	35.4 [26.5, 48.0]	0.16 [0.12, 0.21]	4% (t)	not est.
SeqMoE	DeepSeek-V4-Flash	RTX PRO 6000	66.2	57.4	21.0 ^ℓ	2.73	38.3 [29.1, 51.2]	1.50 [1.12, 1.97]	31% (i)	weak; survives
SeqMoE	GPT-OSS-120B	RTX PRO 6000	27.4	139.2	33.1 ^ℓ	4.21	74.3 [56.5, 99.6]	1.87 [1.40, 2.46]	37% (t)	weak; survives
SeqMoE	Qwen3-30B-A3B FP8	RTX 4090	13.1	104.1	30.9 ^ℓ	3.37	61.7 [46.9, 82.6]	1.69 [1.26, 2.22]	28% (t)	weak; survives
SeqMoE	Qwen3-30B-A3B FP8	RTX 4090	4.4	47.9	21.1 ^ℓ	2.27	48.9 [37.0, 65.7]	0.98 [0.73, 1.29]	13% (t)	weak; not est.
SeqMoE	Qwen3.6-35B-A3B BF16	RTX 5090	25.8	118.3	37.9 ^ℓ	3.12	56.4 [43.0, 75.4]	2.10 [1.57, 2.75]	39% (i)	weak; survives
SeqMoE	Qwen3.6-35B-A3B BF16	RTX 5090	9.7	70.2	28.2 ^ℓ	2.49	46.5 [35.3, 62.3]	1.51 [1.13, 1.99]	33% (i)	weak; survives

most half the predicted equal-memory speed (Table 2). (3) **8 of 22 claimed gains survive** against the predicted baseline; the survivors include designs whose mechanism is not caching at all, such as KTransformers’ CPU kernels. (4) **Systems are far from the physical speed-of-light**: the median adjudicated system reaches 24% of it, and the best 77%.

How much to trust this. The model is conservative for llama.cpp: on our A10 it under-predicts current llama.cpp by 2–35%. Our “weak baseline” calls are therefore conservative, and our “survives” calls generous. Refitting with our first-party runs, a sensitivity analysis we did not register, gives 8 survivors and 17 weak baselines. On the GH200 anchors (Section 5) the model also under-predicts llama.cpp in the offloaded regime; the registered A100 anchors, on the GPU several audited papers used, remain to be run. The audit cannot see implementation quality

inside a system, only its speed against two references.

8 Case Study: A System Against the Bound

We measured our own expert cache with a GPU-signalled CPU hand-off (“mailbox”) on the A10 against llama.cpp `--n-cpu-moe` at equal expert memory, on gpt-oss-20b and -120b (MXFP4) and Qwen3-30B-A3B (Q4_K_M, Q8_0), 12 held-out prompts, first 192 answer tokens after prefilling the whole prompt, on dataset (D) and own (S) text, on one instance with locked clocks.

It is 1.31–1.66 $\times$ faster than llama.cpp across budgets, arms and models (H16, $\geq 1.30\times$ at 25%, holds on both arms), reaching 47–64% of the implementation-relative speed-of-light against 33–45% for llama.cpp. Three observations matter for evaluation rather than for the system.

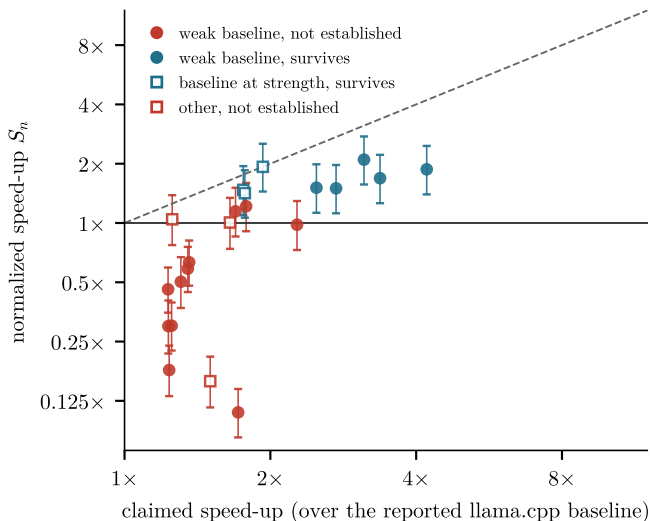


Figure 1: Claimed speed-up (over the row’s strongest reported llama.cpp baseline, else its strongest baseline) against normalised speed-up (over the predicted equal-memory llama.cpp baseline), adjudicated rows, with the band. Below the dashed line the claim shrinks against a strong baseline; below 1 the gain is not established.

The same configuration on a second A10 instance of the same type gave $1.47\times$ instead of $1.63\times$ (gpt-oss-20b) and $1.65\times$ instead of $1.81\times$ (gpt-oss-120b): that instance’s host read 166 instead of 151 GB/s, and llama.cpp’s CPU-bound baseline gained more than the cache did. On the models’ own text (a later job on the same instance) the cache’s speed at 12.5% changed by -6 to -2% and llama.cpp’s by -2 to +9%; the lower own-text speed-ups come mostly from the baseline, and we cannot separate locality from run-to-run drift at this size. And a step-time model of the three systems (our cache, its static-layer variant and llama.cpp), fitted on calibration sequences on the other instance, predicted all 33 configurations per arm with 5.0% median error (maximum 15.0%; H17).

9 Pitfalls We Hit

- **Prefill caps put decode steps in the prompt.** Our earlier harness prefilled at most 128 prompt tokens and teacher-forced the next 192 tokens as decode, so 42% of gpt-oss’s and 32% of Qwen3’s “decode” steps were the user’s own text. On the same instance, fixing it changed speed-ups at 25% by at most $0.15\times$; it mattered more for fidelity checks, where it mixed user-text prediction into loss.
- **Instances of one type are not one machine.** Two A10 VMs differed by 10% in host bandwidth and up to 14% in llama.cpp speed, moving our speed-ups by up to $0.16\times$. Report host bandwidth measured on the instance used.
- **Format is part of the workload.** A reasoning

Table 3: Our cache (“mailbox”) against llama.cpp `--n-cpu-moe` at equal expert memory on one A10 instance, response windows. tok/s on dataset text (D); speed-ups on D and on the models’ own text (S); fractions of the implementation-relative speed-of-light where an all-GPU run exists.

model	budget	llama	cache	$\times D$	$\times S$	cache/SoL	llama/SoL
gpt-oss-20b	12.5%	58.9	86.2	1.46	1.31	48%	33%
gpt-oss-20b	25.0%	65.4	99.3	1.52	1.43	55%	36%
gpt-oss-20b	50.0%	77.7	114.3	1.47	1.44	63%	43%
Qwen3-30B Q4_K_M	12.5%	66.4	96.2	1.45	1.36	49%	34%
Qwen3-30B Q4_K_M	25.0%	73.1	105.9	1.45	1.40	54%	37%
Qwen3-30B Q4_K_M	50.0%	85.7	116.4	1.36	1.40	59%	43%
Qwen3-30B Q8.0	12.5%	47.2	69.4	1.47	1.41	—	—
Qwen3-30B Q8.0	25.0%	47.7	78.9	1.66	1.52	—	—
Qwen3-30B Q8.0	50.0%	61.1	87.9	1.44	1.47	—	—
gpt-oss-120b	12.5%	40.8	63.7	1.56	1.52	—	—
gpt-oss-120b	25.0%	44.5	71.1	1.60	1.61	—	—

model scored on answers without its reasoning is off-distribution (Section 6).

- **Hit rate is not speed.** Our first cache matched the simulator’s hit rates and was no faster than no cache; three coordination costs, not bandwidth, ate the gain.
- **Novelty is cheap to check and expensive to skip.** We built a system before searching; Hybri-MoE, FreeToken, an unnamed CPU-GPU system [8] and four llama.cpp forks and pull requests [15] had converged on the same design.

10 A Reporting Contract in Seconds

Table 4 completes the trace-level contract of [28] with the time-level fields its D1 item leaves open. Every field is cheap: a STREAM run, a copy microbenchmark, the GGUF header, one llama.cpp command line, and a trace of the evaluated text. With them, a reader can compute the speed-of-light and the strong baseline for any result.

11 Limitations

- **The model’s error bounds what the audit can adjudicate.** Effects below about $1.3\times$ are inside the band.
- **Hardware coverage.** Our first-party runs used an A10, an A100 and a GH200 in the cloud; most audited rows are desktop RTX 4090/5090 machines with dual-channel DRAM, which we model but did not measure.
- **Generation checkpoints.** Where the bf16 model did not fit the available GPU, own-text arms were generated (and scored) with the vendor FP8 or on-the-fly FP8 weights; traces use the original weights (bf16, MXFP4 experts for gpt-oss) in fp32 arithmetic.

Table 4: A reporting contract in seconds: the fields that make a batch-1 offloading result checkable against the bound and a strong baseline.

field	report	feeds
Host	CPU, DRAM type/channels/speed, and STREAM Triad (or read) bandwidth at the thread count used	B_c
GPU and link	GPU model and clocks; measured device-read bandwidth; PCIe generation/width and measured pinned H2D	B_g, B_p
Bytes	Model, exact quantisation per tensor class; dense bytes per token and bytes per expert (e.g. from the GGUF header)	D, X
Budget	GPU bytes given to routed experts, per layer or pooled, and what else shares GPU memory	C
Text	Evaluated prompts and answers; whether the answer is the model’s own; full prompt prefilled; decode = answer tokens	trace
Routing	The routing of the evaluated text, released, so M^* can be recomputed	M^*
Strong baseline	llama.cpp commit and full command line, with <code>--n-cpu-moe</code> chosen to fill the same GPU memory	–
Result	Decode tok/s (single request), repeats and spread, and the fraction of the speed-of-light it implies	$\bar{T}$

- **Extraction by agents.** The audit’s extraction and re-check passes were run by AI agents; the protection is that every value carries a verbatim quote and location, and every change is logged, so any row can be checked by hand.
- **Scope of the bound.** Exact routing only: expert skipping, substitution or deferral fall outside it by definition and are flagged in the audit.

12 Conclusion

Stated in seconds and against a baseline configured the way users run it, the literature on MoE offloading looks different: most reported llama.cpp baselines are weak, a minority of claimed gains survive, and the median system reaches 24% of what its hardware physically allows. The bound, the validated model, the traces and the audit set are released so that the next paper can report where it stands in one line.

Acknowledgments. Parts of the code, the experiments and the text were drafted with the help of an AI system (Claude, Anthropic). The author reviewed the work and takes responsibility for its content.

References

[1] Shiyi Cao, Shu Liu, et al. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs.

arXiv preprint arXiv:2411.11217, 2024.

- [2] Chen et al. KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, 2025.
- [3] Liangkun Chen, Zijian Wen, Tian Wu, Xiaoxi Zhang, and Chuan Wu. SP-MoE: Speculative decoding and prefetching for accelerating MoE-based model inference. *arXiv preprint arXiv:2510.10302*, 2025.
- [4] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- [5] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023–2026.
- [6] Torsten Hoefer, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241), 2021.
- [7] Torsten Hoefer and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [8] En-Ming Huang, Li-Shang Lin, et al. Efficient CPU-GPU collaborative inference for MoE-based LLMs on memory-limited systems. *arXiv preprint arXiv:2512.16473*, 2025.
- [9] Ranggi Hwang, Jianyu Wei, et al. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024. arXiv:2308.12066.
- [10] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefer. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- [11] Yinsicheng Jiang, Yao Fu, Yeqi Huang, et al. MoE-CAP: Cost-accuracy-performance benchmarking for mixture-of-experts systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2412.07067.
- [12] Keisuke Kamahori, Tian Tang, et al. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033*, 2024.
- [13] Fangzhou Liang, Yibin Shen, Jianmin Hu, Jiayang Xu, Hanchi Gao, Minxian Xu, and Zili Meng. SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC. *arXiv preprint arXiv:2609.18110*, 2026.
- [14] Yu Lin, Yiming Wang, Runyuan Cai, Hanze Liu, and Xiaodong Zeng. The other half of the memory wall: Serving 35B MoEs from SSD with trained routing prediction. *arXiv preprint arXiv:2609.18063*, 2026.
- [15] llama.cpp contributors. llama.cpp discussion #24528: MoE expert cache benchmarks. <https://github.com/ggml-org/llama.cpp/discussions/24528>, 2026. Accessed September 2026.

- [16] Xinyu Qiu, Chuhong Xu, et al. GGUF-metadata prediction of single-sequence llama.cpp throughput across three systems. *arXiv preprint arXiv:2609.14864*, 2026.
- [17] Yuan Si, Yufeng Lin, et al. Who should own the expert cache? kernel-managed tiering for trillion-parameter MoE inference. *arXiv preprint arXiv:2608.12103*, 2026.
- [18] Xiaoni Song, Zihang Zhong, et al. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024.
- [19] Shriniwas Ramesh Suram. Cacheable by design? training mixture-of-experts routers for locality against the edge memory-bandwidth wall: A pre-registered negative result, with a systems measurement study. *arXiv preprint arXiv:2608.18261*, 2026.
- [20] Peng Tang, Jiacheng Liu, et al. HOBBIT: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433*, 2024.
- [21] Wenfeng Wang, Jiacheng Liu, Xiaofeng Hou, Xinfeng Xia, Peng Tang, Mingxuan Zhang, Chao Li, and Minyi Guo. MoE-SpeQ: Speculative quantized decoding with proactive expert prefetching and offloading for mixture-of-experts. *arXiv preprint arXiv:2511.14102*, 2025.
- [22] Wenxin Wang, Yule Hou, et al. Achieving cloud-grade SLOs for local mixture-of-experts inference through CPU-GPU hybrid design. *arXiv preprint arXiv:2606.10493*, 2026.
- [23] Zihan Wang, Yuqi Wang, et al. SeqMoE: Toward full-load performance via predictive and graph-compatible MoE offloading. *arXiv preprint arXiv:2609.12978*, 2026.
- [24] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [25] Leyang Xue, Yao Fu, et al. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.
- [26] Shuo Yang, Xiaozhe Fan, Melissa Pan, Haocheng Xi, Zhe Wang, Shanlin Sun, Kurt Keutzer, Song Han, Matei Zaharia, Chenfeng Xu, and Ion Stoica. FreeToken: Efficient edge-native MoE serving with bandwidth-adaptive execution. *arXiv preprint arXiv:2608.16157*, 2026.
- [27] Yichao Yuan, Lin Ma, et al. MoE-Lens: Towards the hardware limit of high-throughput MoE LLM serving under resource constraints. *arXiv preprint arXiv:2504.09345*, 2025.
- [28] Yu Zhang. Reproducible evaluation of MoE expert caching: Replay semantics, workload contamination, and operating regimes. *arXiv preprint arXiv:2608.07911*, 2026.
- [29] Shuzhang Zhong et al. HybriMoE: Hybrid CPU-GPU scheduling and cache management for efficient MoE inference. In *Design Automation Conference (DAC)*, 2025. arXiv:2504.05897.
- [30] Shien Zhu, Samuel Bohl, et al. Pre-attention expert prediction and prefetching for mixture-of-experts large language models. *arXiv preprint arXiv:2511.10676*, 2025.

A Audit rows

Table 5: All 52 modelled rows (of the 57 left after the re-check). Budget: GPU bytes for routed experts (n_{cpu}/L MoE layers on the CPU in the predicted baseline; * imputed upper bound). Other columns as in Table 2; SoL_g: the global-budget variant.

system	model	GPU	budget GB (n/L)	sys.	base	claim	pred.	S_n	SoL (SoL _g)	label
CPU-GPU collab- orative inf.	Mixtral-8x7B	RTX 4090	19.7 (25/32)	4.8	4.2	1.14	4.9	0.99	13.1 (14.0) t	n.a. (claim)
CPU-GPU collab- orative inf.	Phi-3.5-MoE	RTX 4090	19.7 (25/32)	10.4	6.3	1.65	10.3	1.01	55.8 (60.9) t	not est.
Fate	Qwen1.5-MoE-A2.7B	RTX 3090	19.3* (6/24)	14.0	7.0	2.00	29.4	0.48	197.9 (197.9) t	n.a. (band)
Fate	Qwen1.5-MoE-A2.7B	GTX 1080 Ti	6.3* (18/24)	3.7	2.6	1.42	18.5	0.20	75.5 (77.6) t	n.a. (band)
Fiddler	Mixtral-8x7B	RTX 6000	19.7 (25/32)	2.5	1.7 ^ℓ	1.49	4.6	0.55	17.3 (18.4) t	n.a. (band)
Fiddler	Mixtral-8x7B	RTX 6000 Ada	44.0 (17/32)	8.3	7.1 ^ℓ	1.17	11.2	0.74	57.4 (57.4) t	n.a. (band)
FreeToken	DeepSeek-V4-Flash	RTX 5090	16.2 (39/43)	24.9	13.0 ^ℓ	1.92	17.8	1.40	80.8 i	n.a. (band)
FreeToken	GLM-5.2	RTX 6000 PRO	58.8* (65/75)	14.9	7.3 ^ℓ	2.04	8.0	1.87	29.8 i	n.a. (band)
FreeToken	Qwen3.6-35B-A3B	RTX 5090	23.8 (26/40)	77.1	42.6 ^ℓ	1.81	35.1	2.19	227.1 i	n.a. (band)
FreeToken	Qwen3.6-35B-A3B	RTX 5090	26.3* (24/40)	73.8	33.0 ^ℓ	2.24	19.1	3.87	204.5 i	n.a. (band)
FreeToken	Qwen3.6-35B-A3B	RTX 4090	18.3* (29/40)	42.9	25.8 ^ℓ	1.66	21.9	1.96	123.1 i	n.a. (band)
FreeToken	Qwen3.6-35B-A3B	RTX 4060 Lap.	4.3* (31/40)	39.3	22.3 ^ℓ	1.76	25.6	1.53	97.3 i	n.a. (band)
HybriMoE	DeepSeek-V2-Lite	RTX A6000	1.9 (20/26)	14.5	11.7 ^ℓ	1.24	80.9	0.18	470.1 (486.4) t	weak; not est.
HybriMoE	DeepSeek-V2-Lite	RTX A6000	5.6 (7/26)	15.3	8.9 ^ℓ	1.72	140.0	0.11	655.0 (655.0) t	weak; not est.
HybriMoE	Mixtral-8x7B	RTX A6000	5.8 (24/32)	4.4	3.5 ^ℓ	1.23	14.6	0.30	53.2 (58.7) t	weak; not est.
HybriMoE	Mixtral-8x7B	RTX A6000	17.4 (8/32)	7.3	7.0 ^ℓ	1.05	26.4	0.28	132.9 (132.9) t	n.a. (claim)
HybriMoE	Qwen2-57B-A14B	RTX A6000	6.4 (21/28)	9.6	7.4 ^ℓ	1.31	19.1	0.50	86.3 (89.5) t	weak; not est.
HybriMoE	Qwen2-57B-A14B	RTX A6000	19.1 (7/28)	9.0	7.2 ^ℓ	1.25	29.8	0.30	115.9 (115.9) t	weak; not est.
KTransformers	DeepSeek-V2.5-1210	A100 40GB	0.0 (59/59)	11.9	7.0 ^ℓ	1.70	10.4	1.15	24.3 i	weak; not est.
KTransformers	DeepSeek-V2.5-1210	RTX 4080	0.0 (59/59)	17.6	9.9 ^ℓ	1.77	12.4	1.42	31.9 i	at strength; survives
KTransformers	DeepSeek-V3-0324	A100 40GB	0.0 (58/58)	5.9	4.7 ^ℓ	1.25	5.6	1.04	12.0 i	at strength; not est.
KTransformers	DeepSeek-V3-0324	RTX 4080	0.0 (58/58)	16.1	9.1 ^ℓ	1.78	13.3	1.22	33.4 i	weak; not est.
KTransformers	Qwen2-57B-A14B	A100 40GB	0.0 (28/28)	22.9	13.0 ^ℓ	1.76	15.5	1.47	35.3 i	at strength; survives
KTransformers	Qwen2-57B-A14B	RTX 4080	0.0 (28/28)	37.0	19.2 ^ℓ	1.93	19.2	1.93	48.1 i	at strength; survives
MoE expert-cache forks	Laguna S 2.1	RTX 5090	26.9* (35/47)	31.3	27.6 ^ℓ	1.13	13.2	2.37	88.2 i	n.a. (band)
MoE-SpeQ	DeepSeek-V2-Lite	A100 40GB	36.3* (0/26)	10.1	7.5	1.35	44.9	0.23	334.8 i	n.a. (band)
MoE-SpeQ	DeepSeek-V2-Lite	A100 40GB	36.3* (0/26)	13.8	9.8	1.41	44.9	0.31	334.8 i	n.a. (band)
MoE-SpeQ	Phi-3.5-MoE	A100 40GB	35.5* (18/32)	6.0	2.6	2.34	10.8	0.56	129.0 (129.0) t	n.a. (band)
MoE-SpeQ	Phi-3.5-MoE	A100 40GB	35.5* (18/32)	6.1	3.4	1.81	10.8	0.56	129.0 (129.0) t	n.a. (band)
MoE-SpeQ	Qwen1.5-MoE-A2.7B	A100 40GB	35.4* (0/24)	13.2	10.3	1.28	59.9	0.22	335.0 i	n.a. (band)
MoE-SpeQ	Qwen1.5-MoE-A2.7B	A100 40GB	35.4* (0/24)	28.3	13.5	2.10	59.9	0.47	335.0 i	n.a. (band)
Pipelined sharding	Qwen3-235B-A22B	RTX 5090	0.0 (94/94)	7.7	5.7 ^ℓ	1.35	13.1	0.59	39.9 i	weak; not est.
Pipelined sharding	Qwen3-235B-A22B	RTX 5090	27.6 (63/94)	11.5	8.5 ^ℓ	1.36	18.2	0.63	107.9 i	weak; not est.
Pipelined sharding	Qwen3-30B-A3B	RTX 5090	0.0 (48/48)	25.7	25.7 ^ℓ	1.00	49.5	0.52	199.1 i	n.a. (claim)
Pipelined sharding	Qwen3-30B-A3B	RTX 5090	5.7 (32/48)	32.1	26.1 ^ℓ	1.23	69.7	0.46	1032.2 (1032.2) t	weak; not est.
Pipelined sharding	Qwen3-30B-A3B	RTX 5070 Ti	13.7 (8/48)	54.9	—	—	90.5	0.61	510.5 (510.5) t	n.a. (no base)
ProMoE	DeepSeek-V2-Lite	RTX 4090	20.3* (8/26)	49.9	45.9 ^ℓ	1.09	19.0	2.63	215.6 (215.6) t	n.a. (band)
ProMoE	Mixtral-8x7B	RTX 4090	21.6* (5/32)	27.0	37.1 ^ℓ	0.73	9.9	2.74	151.5 (151.5) t	n.a. (band)
ProMoE	Qwen1.5-MoE-A2.7B	RTX 4090	19.4* (6/24)	67.2	59.6 ^ℓ	1.13	27.6	2.44	216.2 (216.2) t	n.a. (band)

system	model	GPU	budget GB (n/L)	sys.	base	claim	pred.	S_n	SoL (SoL _g)	label
ProMoE	Qwen2-57B-A14B	RTX 4090	18.2* (10/28)	26.9	23.0 ^ℓ	1.17	13.7	1.96	135.8 (135.8) t	n.a. (band)
SP-MoE	DeepSeek-V2-Lite	A100 40GB	3.3 (24/26)	5.6	3.7	1.50	35.4	0.16	142.0 (146.4) t	not est.
SP-MoE	Mixtral-8x7B	RTX 3090	19.5* (26/32)	1.2	1.1	1.14	5.3	0.23	15.5 (16.5) t	n.a. (band)
SP-MoE	Mixtral-8x7B	RTX 4090	19.5* (26/32)	1.3	1.1	1.17	4.4	0.29	13.4 (14.3) t	n.a. (band)
SP-MoE	Mixtral-8x7B	A100 40GB	35.5* (20/32)	1.5	1.3	1.11	5.9	0.25	32.2 (36.7) t	n.a. (band)
SP-MoE	Phi-3.5-MoE	RTX 4090	19.5* (25/32)	3.3	2.8	1.18	9.5	0.35	56.9 (62.0) t	n.a. (band)
SeqMoE	DeepSeek-V4-Flash	RTX PRO 6000	66.2 (24/43)	57.4	21.0 ^ℓ	2.73	38.3	1.50	183.2 i	weak; survives
SeqMoE	GPT-OSS-120B	RTX PRO 6000	27.4 (20/36)	139.2	33.1 ^ℓ	4.21	74.3	1.87	377.9 (377.9) t	weak; survives
SeqMoE	Qwen3-30B-A3B FP8	RTX 4090	13.1 (27/48)	104.1	30.9 ^ℓ	3.37	61.7	1.69	373.7 (373.7) t	weak; survives
SeqMoE	Qwen3-30B-A3B FP8	RTX 4090	4.4 (41/48)	47.9	21.1 ^ℓ	2.27	48.9	0.98	373.7 (373.7) t	weak; not est.
SeqMoE	Qwen3.6-35B-A3B BF16	RTX 5090	25.8 (24/40)	118.3	37.9 ^ℓ	3.12	56.4	2.10	305.1 i	weak; survives
SeqMoE	Qwen3.6-35B-A3B BF16	RTX 5090	9.7 (34/40)	70.2	28.2 ^ℓ	2.49	46.5	1.51	211.5 i	weak; survives
llama.cpp 27861	PR Qwen3-30B-A3B deriva	Radeon RX 7600	4.4 (36/48)	16.5	14.4 ^ℓ	1.15	23.6	0.70	168.6 (168.6) t	n.a. (claim)

Not modelled: `promoe-llamacpp-deepseekmoe16b-fp16-rtx4090-b1` (architecture needs remote code (not in transformers)); `fate-deepseekmoe16b-rtx3090-b1` (architecture needs remote code (not in transformers)); `fate-deepseekmoe16b-gtx1080ti-b1` (architecture needs remote code (not in transformers)); `lcpp-pr27861-sdroege-qwen3.8-flash-next-q4kx1-r9700` (per-layer n-gram lookup table is not a routed expert or dense weight in the model); `lcpp-pr27861-sirfyy-qwen3.8-flash-next-q4kx1-rtxpro4500` (per-layer n-gram lookup table is not a routed expert or dense weight in the model).

B Pre-registration log

id	phase	registered claim	outcome
H0	1 (A10)	llama.cpp never beats the roofline	holds, 31/31
H1	1	ncu DRAM bytes within $\pm 10\%$ of GGUF bytes	fails: +12–15% (inline ECC over-read)
H2	1	M4 median APE $\leq 20\%$	fails: 27.4% (every prediction too slow)
H3	1	1/tok _s affine in --n-cpu-moe, $R^2 \geq 0.98$	holds, 0.9965–0.9987
H4	1	intercept/slope decomposition	intercept over-predicted 25–55%
H5	1b	two-run calibration	not run
H1'	1b	counter calibration with ncu	not run
H6	2	cache top-1 agreement $\geq 98\%$ vs all-CPU	fails narrowly: 97.4–97.9%
H7	2	cache $\geq 1.2\times$ llama.cpp at 25%	fails: cache ran at all-CPU speed
H9	2	measured hit rates within ± 5 points of simulated	holds on job 009 (0.679 vs 0.677)
H8, H10	2	prediction; policy ordering	not scored: system redesigned (phase 3)
H11	3	mailbox $\geq 1.30\times$ llama.cpp at 25%	holds, 1.50–1.81 $\times$ (windows defect, see H16)
H12	3	hand-off alone $\geq 1.05\times$	fails on Q8_0 (1.022, 1.047 $\times$)
H13	3	NLL within 1%, top-1 $\geq 95\%$	fails on gpt-oss-120b (89.5–90.1%)
H14	3	step-time model median APE $\leq 10\%$	holds, 2.9%
H15	3	own text: $\geq 1.30\times$	holds, 1.54–1.94 $\times$
F1–F4	4	provenance changes a conclusion	none fires on any model
P1	4	own text has lower NLL	holds on every model
P2	4	greedy text is more reused	fails on 3 of 4
P3	4	$ \Delta\text{DFA} \geq 2$ points on gpt-oss	holds
P4	4	DFA beats LRU in S	fails on Qwen3 (both arms)
4.4	4	classify the 120b anomaly	class 3, format
H16	4.5	$\geq 1.30\times$ at 25%, correct windows	holds, 1.31–1.66 $\times$ over all budgets
H17	4.5	step model within 10% median	holds, 5.0% (max 15.0%)
P5	4.6	$\geq 1/3$ of llama.cpp baselines weak	holds, 16 of 20
P6	4.6	median system $\leq 60\%$ of SoL	holds, 24%
A1	4.7	no anchor beats the floor	holds (GH200)
A2, A3	4.7	A100 anchors	not run: no A100 capacity

Every registration is a section of `prereg/PROTOCOL.md` committed before the runs it concerns; the deviation logs there record every change.